

DATA STRUCTURE LAB

Array ArrayList

// [] [] [] [] [] [] [] [] [] [] [] [] [] [] [] []

length

□ □ □ □ □

size ≠ capacity

□ □ □ □ □ □ □ □ ≠ □ □ □ □ □



API

```
1 int[] scores = {85, 92, 78};  
2 scores[1] = 95;  
3 int n = scores.length;
```

NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH
[] · NO GRAPH
NO GRAPH
NO GRAPH
NO GRAPH

```
1 List<Integer> scores = new ArrayList<>();  
2 scores.add(85);  
3 scores.set(0, 95);  
4 int n = scores.size();
```

NO GRAPH
NO GRAPH
API · □□□□□

size NO GUPH capacity ☐☐☐☐☐☐

10

20

30

☐

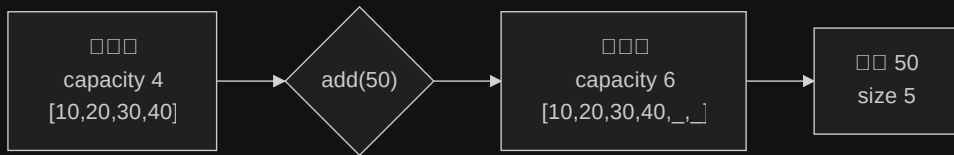
size = 3

☐☐☐☐☐☐☐☐

capacity = 4

☐☐☐☐☐☐☐☐

capacity NO GUPH NO GUPH NO GUPH NO GUPH NO GUPH NO GUPH NO GUPH List API ☐☐☐☐☐☐



4 → 6
 $4 + \text{floor}(4 / 2)$

4
add $O(n)$

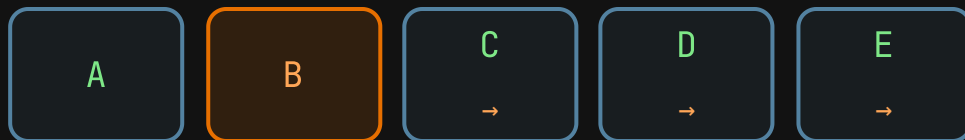
$O(1)$
□□□□□□

OpenJDK 1.5× List □□□□



```
1 List<Order> orders = new ArrayList<>(incoming);
```

```
□□□□□□□□□□□□□□□□□□□□□□
```



add

□□□□□□□□

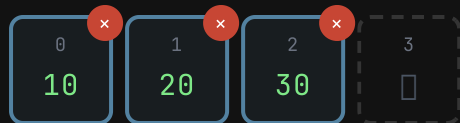
index 1 □□

□□□□□□□□ $O(n)$



Array length

ArrayList size / capacity



size = 3 · capacity = 4 · 0 · 0

40

add

1

insert

capacity add insert remove

primitive wrapper

```
1 int[] raw = {1, 2, 3};  
2 // 000 int
```

Array NOT ES6 NOT ES6 NOT ES6 NOT ES6 primitive NOT ES6 int NOT ES6 double NOT ES6 char

```
1 List<Integer> boxed = new ArrayList<>();  
2 boxed.add(1); // int → Integer  
3 int n = boxed.get(0); // Integer → int
```

NOT ES6 NOT ES6 NOT ES6 NOT ES6 NOT ES6 NOT ES6 List<int> NOT ES6 NOT ES6 wrapper NOT ES6
autoboxing



```
1 List<Integer> values = new ArrayList<>();
2 values.add(null);
3
4 Integer boxed = values.get(0); // boxed == null
5 int raw = values.get(0);      // NullPointerException
6
7 Integer maybe = values.get(0);
8 if (maybe != null) {
9     int safe = maybe;
10    System.out.println(safe);
11 }
```

values.get() returns Integer → int

Arrays.asList

```
1 List<String> readOnly = List.of("Ada", "Linus");  
2 // set / add
```

```
Arrays.asList().view().asArrayList()
```



```
import java.util.*;
```

```
class Roster {  
    public static void main(String[] args) {  
        List<String> names =  
            new ArrayList<>(List.of("Ada", "Linus"));  
        names.add("Grace");  
        System.out.println(names);  
    }  
}
```

□□

1. `String[]`
2. `ArrayList`
3. `Monaco` `Java`



□□	Array	ArrayList
<code>get(index)</code>	$O(1)$	$O(1)$
<code>set(index)</code>	$O(1)$	$O(1)$
NO ERROR NO ERROR <code>add</code>	□□□	NO ERROR NO ERROR $O(1)$
NO ERROR NO ERROR NO ERROR NO ERROR NO ERROR <code>//</code> NO ERROR NO ERROR	NO ERROR NO ERROR NO ERROR NO ERROR $O(n)$	$O(n)$
□□	□□	NO ERROR NO ERROR $O(n)$

Big-O □□□□□□boxing□□□□□□□□□□□□



□□□□□□

  7  primitive □□

double[]

□□□□

□□□□□□□

ArrayList

  8 × 8

□□□□□□□□

char[][]

□□□□

□□□□□□

ArrayList

   primitive   Array            ArrayList□



size ☐☐☐☐☐ capacity ☐☐☐☐☐

☐ ☐ ☐ 1.5x ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

primitive ☐ ☐ ☐ ☐ ☐ ☐ ☐ API ☐☐☐

